

Plan Audit Certificates as the RPLAN/RCTX Fixed Point

Gio Della-Libera

Draft — May 2026

Abstract

This paper presents RPLAN, RCTX, audit certificates, manifests, and verifier integration as the BISECT fixed point for heterogeneous redistricting algorithms. It is an audit/certification infrastructure paper: certificates verify declared profiles, canonical ordering, hashes, package integrity, and lineage. They do not certify political fairness, legal sufficiency, or optimality beyond the recorded profile and attached evidence.

1 Introduction

U.20 documents `rplan-core`, `rplan-io`, `rplan-audit`, `rplan-cli`, and the BISECT manifest verifier. The paper asks whether BISECT can use a neutral plan/context/certificate family as the fixed point for construction, exact, local-search, Pareto, and future algorithm families.

The fixed point is not another search method. It is the acceptance contract at the end of a method run. A construction algorithm may export a deterministic baseline package, an exact method may export a solve report and final plan, a large-neighborhood search may export a parent/child improvement record, and a Pareto search may export a selected frontier row. U.20 says what all of those outputs must become before they are carried into reports: a plan, a context, a profile-scoped certificate, a manifest, and a lineage record with stable hashes.

This narrowness is the core design choice. RPLAN/RCTX does not erase the method-specific evidence from T.14–T.17 or U.16–U.19. Instead, it gives those families one common final-plan surface so that BISECT can verify package integrity, context agreement, declared profile checks, and provenance without pretending to re-prove every upstream algorithmic claim.

2 Schema Boundary

RPLAN stores plan identity and assignments. RCTX stores graph, population, canonical unit order, and context needed for verification. Certificates and manifests record declared profiles, verifier results, hashes, lineage, and algorithm metadata.

Artifact	Owens	Must not own
RPLAN	final assignments and plan identity	graph/population context
RCTX	units, graph, population, canonical order	algorithm-specific proof
Certificate	profile checks, verifier results, lineage	legal compliance claim
Manifest	files, hashes, package inventory	substantive validity by itself

Table 1: RPLAN/RCTX fixed-point artifact responsibilities.

The fixed point is a shared final-plan contract, not a new algorithmic claim. Construction, exact optimization, local search, and Pareto comparison may carry different upstream evidence, but their final packages must agree on the same artifact vocabulary.

The atlas framing requires both positive and negative verifier paths to be concrete. A passing package shows the plan, context, certificate, manifest, and hash chain agreeing. A failing package should identify the exact broken invariant: missing context, order mismatch, stale hash, unsupported profile, or broken lineage pointer. That failure reason is part of the audit product rather than an implementation footnote.

Invariant	Required check	Failure mode
Canonical unit order	RPLAN assignments align with RCTX units	order mismatch
Hash integrity	manifest hashes match files	tamper or stale file
Profile scope	certificate names declared checks	unsupported claim
Lineage	algorithm/run ids link to parent artifacts	broken provenance

Table 2: Fixed-point invariants for verifier-facing packages.

The common package shape is deliberately small. A typical public package has a plan file, a context file, an audit-certificate JSON file, and a manifest JSON file. Method-specific files may sit beside those artifacts, but the verifier-facing contract is the same: the manifest names the files and hashes, the certificate binds the plan and context under a declared profile, and the lineage record says which algorithm run produced the exported plan.

3 Audit Semantics

The verifier contract is profile-scoped. It can confirm that declared constraints, lineage, canonical ordering, hashes, and package contents match the profile. It cannot certify fairness, neutrality, or legal sufficiency outside the declared audit profile and available context.

Solver reports, heuristic transcripts, and selected-frontier manifests are upstream evidence. The certificate consumes their final exported artifacts and checks the package under a declared profile. This keeps exact proofs, heuristic-improvement summaries, and Pareto-selection records from being collapsed into one ambiguous claim.

Family	Example package	Evidence retained before U.20
Construction	T.14–T.17 golden packages	construction summaries, witnesses, and final assignments
Exact search	U.16 and U.17 packages	solve reports, separation/column events, and model-relative certificates
Local search	U.18 package	parent plan, candidate move, child plan, and improvement status
Pareto search	U.19 selected-frontier package	frontier row, selected index, objectives, and export metadata
Audit benchmark	U.20 grid packages	manifest, certificate, context, hashes, and negative fixture catalog

Table 3: Public package families currently exercising the fixed point.

A successful verification should therefore be read as: this plan, with this context, under this profile, with these hashes and lineage records, passed the declared checks. It should not be read as: this plan is fair, legally adequate, or preferred among all available alternatives.

The public verifier workflow is intentionally fixed point shaped: algorithm output becomes a package, the package certificate is checked, and the checked package can then be referenced by reports. The neutral surface is the RPLAN certificate verifier; the BISECT manifest verifier consumes the same package manifest shape as a bridge.

```
rplan-cli verify-certificate \
  --certificate audit-certificate.json \
  --plan plan.rplan \
  --context context.rctx

bisect-cli verify --manifest manifest.json
```

The first command checks the certificate against the plan and context. The second command checks the BISECT package manifest bridge. In the public examples these commands are intentionally run on small packages so that a reader can inspect every file and every hash involved in the decision.

4 Evaluation Plan

Evidence includes RPLAN crate tests, certificate-verifier tests, BISECT bridge tests, final-plan sidecars across construction, exact, local-search, and Pareto families, public golden packages, a U.20 grid10 audit benchmark package, and a tamper/failure-mode catalog.

The evaluation standard is therefore mechanical rather than rhetorical. A method earns the right to appear in downstream reports only after its exported package survives manifest checks, certificate checks, and the negative fixture catalog. If a package lacks context,

Fixture	Expected verifier signal	Meaning
Missing context	certificate requires context hash	contextual claims need the RCTX, not only the RPLAN
Plan tamper	certificate plan hash mismatch	assignment content changed after certification
Context tamper or stale context	certificate context hash mismatch	graph, population, order, or source content changed
Canonical unit-order mismatch	certificate context hash mismatch	positional assignments no longer match the certified unit order
Profile mismatch	legal profile chamber mismatch or RPLAN legal profile mismatch	the certificate cannot be reused for a different declared profile
Reserved lineage field	algorithm lineage extra uses reserved certificate field	method metadata cannot override top-level certificate semantics

Table 4: Verifier-facing negative fixtures for the U.20 contract.

Claim	Current evidence	Publication gap
RPLAN/RCTX separate plan identity from context	rplan-core and rplan-io tests plus public schema examples	larger real-data packages
Certificates verify declared profiles and package integrity	rplan-audit, rplan verify-certificate , public 3x3 package tests, and the grid10 audit benchmark package	Additional manifest/certificate snippets
Algorithm families converge to one final-plan path	Final-plan sidecar tests across T/U papers and public packages by family	larger method-produced packages
Verifier catches tamper and mismatch failures	CLI tests for missing context, plan tamper, context hash changes, unit-order mismatch, profile mismatch, stale context, missing-input results, and lineage-reserved fields	more adversarial fixture variants

Table 5: Claim and evidence ladder for U.20.

Corpus slice	Public package role	What the package proves
T.14 spectral-style construction	construction export	a constructed final assignment can enter the common package path
U.16 branch-and-cut	exact-search export	a model-relative solve artifact can hand off a final RPLAN/RCTX package
U.18 large-neighborhood search	local-search export	a parent/child improvement record can end in the same audit surface
U.19 selected frontier	Pareto export	a chosen frontier row can be separated from the certified plan package
U.20 grid3x3 reference	certificate example	plan, context, certificate, and manifest hashes agree on an inspectable fixture
U.20 grid10 benchmark	benchmark package	the fixed point can be timed and regression-tested on a larger synthetic grid

Table 6: Public corpus used to check that U.20 is a cross-family endpoint.

mutates a plan, changes a context hash, changes unit order, reuses the wrong profile, or smuggles reserved lineage fields into method metadata, the verifier must reject it with a specific reason.

5 Limitations

Certificates verify a profile, not all law. Some constraints require data that may not be present in every RCTX. External tools must preserve canonical unit ordering, hashes, and package semantics for interoperation to remain valid. The fixed point also depends on honest profile naming: a package can be valid for one profile while lacking the context required by another.

Publication-ready evidence now has an initial public 3x3 package, a grid10 audit benchmark package, public construction/exact/local-search/Pareto package families, and CLI negative tests for missing context, plan tamper, context hash changes, unit-order mismatch, stale context, profile mismatch, missing-input constraints, and reserved lineage fields. The next evidence ceiling is not the fixed-point contract itself but breadth: larger real-data packages, more adversarial fixture variants, solver-version transcripts, and independent package consumers outside this repository.

6 Conclusion

U.20 is the fixed-point paper for the roadmap. It explains how heterogeneous algorithm families can converge on a common plan/context/audit package while keeping their method-specific evidence and limitations explicit. That closes the T/U implementation arc in a precise way: construction, exact optimization, local search, and Pareto selection may disagree about how candidate plans are found, but they now agree on what a final plan must carry before BISECT treats it as auditable.

The invariant is intentionally narrow. Certificates are useful because they are profile-scoped and tied to verifiable package semantics rather than broad legal or political conclusions. U.20 therefore turns the algorithm family into an inspectable artifact pipeline: method evidence first, fixed-point package next, downstream report last.

Certified recursive bisection adds a model-optimality layer above this fixed point. The recursive tree proves which assignment the enacted optimization selects; U.20 then verifies that the emitted RPLAN/RCTX package faithfully contains that assignment and satisfies its declared audit profile. Neither artifact substitutes for the other.

References

BISECT project. U.20 plan audit certificates paper plan, 2026. Local planning document and RPLAN schema roadmap.